Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №6

“ Автоматизоване тестування Python-програм ”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета роботи: сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest, навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, застосовувати mocks і аналізувати test coverage.

1. Номер індивідуального варіанта

Варіант № 10: «Система обліку складських запасів».

2. Короткий опис тестованого проєкту

Тестований проєкт являє собою систему обліку складських запасів, яка забезпечує:

- Моделювання товарних позицій (WarehouseItem) з валідацією атрибутів (код, назва, кількість, ціна) та обчисленням загальної вартості;
- Керування складськими запасами (Warehouse) — надходження товарів, списання з перевіркою дефіциту (InsufficientStockError), пошук товарів за унікальним кодом;
- Універсальне сховище даних у пам'яті (InMemoryRepository[T]);
- Бізнес-сервіси для обчислення загальної вартості складу, визначення товарів із низьким залишком (LOW_STOCK_THRESHOLD), а також асинхронну взаємодію з постачальником для автозамовлення (AsyncReorderService);
- Потоковий імпорт даних з CSV-файлів із фільтрацією/обробкою некоректних записів (malformed records) та експорт стану складу у формат JSON;
- Гнучке налаштування конфігурації через змінні середовища.

3. Структура production code

Код застосунку розташований у каталозі src/warehouse/:

src/warehouse/

├── __init__.py # Ініціалізація пакета та експорт публічного API

├── models.py # Доменні моделі (WarehouseItem, Warehouse) та винятки (InsufficientStockError, InvalidItemError)

├── config.py # Зчитування конфігурації (LOW_STOCK_THRESHOLD) із змінних середовища

├── repositories.py   # Універсальне сховище в пам'яті (InMemoryRepository)

├── services.py   # Бізнес-сервіси (WarehouseService, AsyncReorderService) та інтерфейс SupplierClient

└── io.py   # Потоковий CSV-імпорт (stream_import_csv) та JSON-експорт (export_to_json)

4. Структура test suite

Тестовий набір розміщено у каталозі tests/ з чітким поділом на модульні та інтеграційні тести:

tests/

├── conftest.py   # Спільні фікстури (sample_item, sample_item_list, sample_warehouse, csv_factory)

├── unit/   # Модульні тести

│   ├── test_models.py   # Тести доменних моделей WarehouseItem та Warehouse

│   ├── test_services.py   # Тести бізнес-сервісів, Mock, AsyncMock, Repository

│   └── test_io_config.py   # Тести IO-операцій (CSV/JSON), tmp_path, monkeypatch

└── integration/   # Інтеграційні тести

    └── test_flows.py   # Повний потік: CSV -> Warehouse -> Operations -> JSON Export

5. Test matrix (Матриця тестування)

Компонент / Модуль	Сценарії тестування	Використані засоби pytest
WarehouseItem	Коректне створення, обчислення total_price, від'ємна ціна/кількість, неправильні типи даних, порожні рядки	@pytest.mark.parametrize, pytest.raises, pytest.approx

Компонент / Модуль	Сценарії тестування	Використані засоби pytest
Warehouse	Додавання товарів, надходження, списання, замовлення понад залишок, пошук за кодом	fixtures, pytest.raises (InsufficientStockError)
CSV Import & JSON Export	Потокове зчитування з CSV, обробка некоректних рядків, збереження у JSON	tmp_path, yield fixture, custom exceptions
Configuration	Зчитування дефолтного та зміненого порогу WAREHOUSE_LOW_STOCK, некоректні типи	monkeypatch.setenv, monkeypatch.delenv
Repository & Supplier	Додавання/пошук об'єктів, підміна зовнішнього постачальника, виклики з помилкою	Mock, return_value, side_effect, interaction assertions
Async Reorder	Асинхронне формування та відправка замовлення постачальнику	AsyncMock, pytest.mark.asyncio, assert_awaited_once_with
Integration Flow	Повний цикл: CSV-імпорт -> оновлення складу -> списання -> перевірка балансу -> JSON-експорт	tmp_path, monkeypatch, комплексні інтеграційні тести

6. Приклади unit tests

```
# tests/unit/test_models.py

def test_warehouse_item_creation_and_total_price(sample_item):
    """Перевірка коректного створення товару та обчислення загальної вартості."""
    assert sample_item.code == "A100"
```

```
    assert sample_item.name == "Laptop"
    assert sample_item.quantity == 10
    assert sample_item.unit_price == pytest.approx(1255.0)
    assert sample_item.total_price == pytest.approx(12550.0)

def test_warehouse_deposit_and_withdraw(sample_warehouse):
    """Перевірка операцій надходження та списання товарів."""
    sample_warehouse.deposit("A100", 5)
    assert sample_warehouse.get_item("A100").quantity == 15

    sample_warehouse.withdraw("A100", 3)
    assert sample_warehouse.get_item("A100").quantity == 12
```

7. Приклад integration test

```
# tests/integration/test_flows.py
@pytest.mark.integration
def test_full_warehouse_lifecycle_flow(tmp_path, monkeypatch):
    """Інтеграційний тест: CSV -> Склад -> Списання -> Перевірка вартості -> Експорт JSON."""
    monkeypatch.setenv("WAREHOUSE_LOW_STOCK", "5")

    csv_file = tmp_path / "incoming_stock.csv"
    csv_file.write_text(
        "code,name,quantity,unit_price\n"
        "ITEM01,Monitor,10,200.0\n"
        "ITEM02,Keyboard,2,50.0\n"
    )

    warehouse = Warehouse()
```

```
    items = list(stream_import_csv(csv_file))
    for item in items:
        warehouse.add_item(item)

    warehouse.withdraw("ITEM01", 8)  # Залишилося 2 (Low stock!)

    assert len(warehouse.get_low_stock_items(threshold=5)) == 2

    json_file = tmp_path / "export_stock.json"
    export_to_json(warehouse, json_file)

    assert json_file.exists()
    assert "ITEM01" in json_file.read_text()
```

8. Приклад fixture

```
# tests/conftest.py
@pytest.fixture
def sample_item():
    """Фікстура для створення поодинокого екземпляра товару."""
    return WarehouseItem(code="A100", name="Laptop", quantity=10,
unit_price=1255.0)
```

9. Приклад fixture із yield

```
# tests/conftest.py
@pytest.fixture
def csv_factory(tmp_path):
    """Фікстура з teardown (yield) для динамічного створення тестових CSV-
файлів."""
```

```
    created_files = []

    def _create_csv(filename: str, content: str):
        file_path = tmp_path / filename
        file_path.write_text(content, encoding="utf-8")
        created_files.append(file_path)
        return file_path

    yield _create_csv

    # Teardown logic
    for file_path in created_files:
        if file_path.exists():
            file_path.unlink()
```

10. Приклад parametrization

```
# tests/unit/test_models.py
@pytest.mark.parametrize("quantity, unit_price", [
    (0, 100.0),      # Нульова кількість
    (1, 0.0),        # Нульова ціна
    (100, 1255.50),  # Валідні значення
    (50, 0.01)       # Граничне мале значення ціни
])
def test_valid_item_parameters(quantity, unit_price):
    item = WarehouseItem(code="TEST", name="Test Item", quantity=quantity, unit_price=unit_price)
    assert item.quantity == quantity
    assert item.unit_price == pytest.approx(unit_price)
```

11. Приклад boundary test

```
# tests/unit/test_models.py

@pytest.mark.parametrize("invalid_quantity", [-1, -100])

def test_negative_quantity_boundary(invalid_quantity):

    """Перевірка некоректної від'ємної кількості на межі допустимих значень."""

    with pytest.raises(InvalidItemError):

        WarehouseItem(code="ERR", name="Bad Quantity",
quantity=invalid_quantity, unit_price=10.0)
```

12. Приклад pytest.raises

```
# tests/unit/test_models.py

def test_withdraw_insufficient_stock_raises_exception(sample_warehouse):

    """Перевірка виклику InsufficientStockError при спробі списати більше, ніж
є."""

    with pytest.raises(InsufficientStockError) as exc_info:

        sample_warehouse.withdraw("A100", 100) # Є лише 10


    assert "Requested 100, but only 10 available" in str(exc_info.value)
```

13. Приклад pytest.approx

```
# tests/unit/test_models.py

def test_total_price_float_precision():

    """Перевірка обчислення ціни з плаваючою крапкою через pytest.approx."""

    item = WarehouseItem(code="F1", name="Float Item", quantity=3,
unit_price=33.3333)

    assert item.total_price == pytest.approx(99.9999, rel=1e-4)
```

14. Приклад tmp_path

```
# tests/unit/test_io_config.py
def test_json_export_writes_correct_file(tmp_path, sample_warehouse):
    """Перевірка експорту у файл через ізольовану тимчасову директорію tmp_path."""
    target_json = tmp_path / "subfolder" / "output.json"
    target_json.parent.mkdir()

    export_to_json(sample_warehouse, target_json)

    assert target_json.exists()
    assert target_json.stat().st_size > 0
```

15. Приклад monkeypatch

```
# tests/unit/test_io_config.py
def test_low_stock_threshold_override(monkeypatch):
    """Перевірка підміни змінної середовища через monkeypatch."""
    monkeypatch.setenv("WAREHOUSE_LOW_STOCK", "15")
    config = get_warehouse_config()
    assert config.low_stock_threshold == 15
```

16. Приклад Mock

```
# tests/unit/test_services.py
def test_supplier_service_with_mock():
    """Перевірка взаємодії з мок-об'єктом постачальника."""
    mock_supplier = Mock(spec=SupplierClient)
    mock_supplier.place_order.return_value = "ORDER-999"
```

```
    service = ReorderService(supplier_client=mock_supplier)
    order_id = service.reorder_item("A100", 50)

    assert order_id == "ORDER-999"
```

17. Приклад return_value

```
# tests/unit/test_services.py
def test_repository_get_returns_mocked_item():
    mock_repo = Mock()
    expected_item = WarehouseItem("X1", "Mocked Item", 5, 10.0)
    mock_repo.get_by_code.return_value = expected_item

    result = mock_repo.get_by_code("X1")
    assert result == expected_item
```

18. Приклад side_effect

```
# tests/unit/test_services.py
def test_supplier_service_handles_failure_side_effect():
    """Перевірка симуляції збою зовнішього сервісу через side_effect."""
    mock_supplier = Mock(spec=SupplierClient)
    mock_supplier.place_order.side_effect = ConnectionError("Supplier API
Unavailable")

    service = ReorderService(supplier_client=mock_supplier)

    with pytest.raises(ConnectionError) as exc_info:
        service.reorder_item("A100", 10)
    assert "Supplier API Unavailable" in str(exc_info.value)
```

19. Interaction assertion

```
# tests/unit/test_services.py
def test_supplier_interaction_assertions():
    mock_supplier = Mock(spec=SupplierClient)
    service = ReorderService(supplier_client=mock_supplier)

    service.reorder_item("ITEM_XYZ", 25)

    # Перевірка фактів та параметрів виклику
    mock_supplier.place_order.assert_called_once_with(item_code="ITEM_XYZ", quantity=25)
```

20. Приклад AsyncMock

```
# tests/unit/test_services.py
@pytest.mark.asyncio
async def test_async_reorder_provider():
    """Перевірка асинхронного автозамовлення з використанням AsyncMock."""
    async_supplier_mock = AsyncMock()
    async_supplier_mock.async_place_order.return_value = "ASYNC-ORDER-123"

    async_service = AsyncReorderService(provider=async_supplier_mock)
    result = await async_service.process_reorder("A100", 20)

    assert result == "ASYNC-ORDER-123"
    async_supplier_mock.async_place_order.assert_awaited_once_with("A100", 20)
```

21. Результат запуску tests

Запуск проводився у консолі за допомогою команди:

pytest -v --cov=warehouse --cov-branch --cov-report term-missing --cov-report=html

22. Кількість passed/failed tests

- Усього тестів: 48
- Успішно пройшли (PASSED): 48
- Провалено (FAILED): 0
- Статус виконання: 100% SUCCESS

23. Coverage report (Покриття коду)

Сформований консольний звіт pytest-cov:

```
Name                       Stmts   Miss Branch BrPart  Cover   Missing
----------------------------------------------------------------------
src/warehouse/__init__.py      5      0      0      0   100%
src/warehouse/config.py       12      0      2      0   100%
src/warehouse/io.py           28      4      8      0    89%   19-20, 23-24
src/warehouse/models.py       38      0     12      0   100%
src/warehouse/repositories.py  8      0      0      0   100%
src/warehouse/services.py     16      0      4      0   100%
----------------------------------------------------------------------
TOTAL                        107      4     26      0  96.99%
```

24. Branch coverage (Покриття гілок)

- Загальна кількість гілок у коді (*Branches*): 26
- Частково покритих гілок (*Partial Branches*): 0
- Загальне покриття коду з урахуванням гілок розгалуження: 96.99% (що суттєво перевищує встановлений поріг у 80%).

25. Фрагмент HTML coverage report

Після запуску було згенеровано детальний HTML-звіт у директорії htmlcov/index.html. Фрагмент підсумкової таблиці з htmlcov/index.html:

```
<table class="index" data-sortable>
    <thead>
        <tr class="tablehead" data-sort-method="number">
            <th class="name left" data-sort-default>Module</th>
            <th>statements</th>
            <th>missing</th>
            <th>excluded</th>
            <th>branches</th>
            <th>partial</th>
            <th class="right" data-sort-datatype="number">coverage</th>
        </tr>
    </thead>
    <tbody>
        <tr class="file">
            <td class="name left"><a
href="z_src_warehouse_models_py.html">src/warehouse/models.py</a></td>
            <td>38</td><td>0</td><td>0</td><td>12</td><td>0</td>
            <td class="right" data-ratio="50 50">100%</td>
        </tr>
        <tr class="file">
            <td class="name left"><a
href="z_src_warehouse_io_py.html">src/warehouse/io.py</a></td>
            <td>28</td><td>4</td><td>0</td><td>8</td><td>0</td>
            <td class="right" data-ratio="32 36">89%</td>
        </tr>
    </tbody>
    <tfoot>
```

```
      <tr class="total">
        <td class="name left">Total</td>
        <td>107</td><td>4</td><td>0</td><td>26</td><td>0</td>
        <td class="right" data-ratio="129 133">96.99%</td>
      </tr>
    </tfoot>
</table>
```

26. Аналіз недостатньо протестованих paths

Згідно зі звітом покриття, не покритими лишилися рядки 19–20 та 23–24 у модулі src/warehouse/io.py:

- Рядки 19–20: гілка обробки винятку ValueError, коли значення ціни unit_price у CSV-файлі містить некоректний тип (наприклад, рядок "bad_price").
- Рядки 23–24: гілка обробки створення об'єкта товару із порожнім кодом або назвою під час CSV-імпорту.

Для досягнення 100% покриття необхідно додати 2 модульних тести у test_io_config.py із CSV-даними, які містять нечислову ціну та порожні поля code/name.

27. Опис знайдених tests реальних defects

Під час розробки та виконання тестів було виявлено 2 дефекти:

1. Дефект 1 (Некоректний розрахунок залишку): У методі withdraw() початково була відсутня перевірка на спробу списання від'ємної кількості товарів (наприклад, withdraw(-5)), що призводило до несанкціонованого збільшення залишків. Написаний boundary-тест виявив цей дефект, після чого у метод було додано відповідну валідацію.
2. Дефект 2 (Типи даних у CSV): При імпорті з CSV цілі числа кількості товарів імпортувалися як рядки ("10" замість 10), що спричиняло помилки під час виконання математичних операцій добавлення. Тест test_csv_import_data_types підсвітив цю проблему, й у модулі io.py було додано явне приведення типів int(row['quantity']).

28. Відповіді на контрольні запитання

1. Що таке automated test?
   Це програма/скрипт, який автоматично виконує код застосунку, подає тестові вхідні дані та порівнює фактичний результат із очікуваним.
2. Які основні переваги автоматизованого тестування?
   Швидкість виконання, висока точність, можливість частішого запуску, виявлення регресій та автоматизація у CI/CD.
3. Що таке regression?
   Це поява помилок у вже працюючому функціоналі після внесення нових змін або рефакторингу коду.
4. Що таке unit test?
   Тест, що перевіряє найменшу ізольовану частину коду (функцію, метод або клас) окремо від інших компонентів.
5. Що таке integration test?
   Тест, що перевіряє коректність спільної роботи кількох модулів, класів або зовнішніх систем (баз даних, файлової системи).
6. Чим integration test відрізняється від unit test?
   Unit-тести ізольовані й швидкі; integration-тести перевіряють взаємодію реальних компонентів і можуть використовувати зовнішні ресурси.
7. Що показує test pyramid?
   Співвідношення кількості тестів різних рівнів: багато юніт-тестів в основі, менше інтеграційних і найменше E2E/UI тестів на вершині.
8. Чому unit tests зазвичай більше?
   Вони дешеві в розробці, виконуються за мілісекунди та дають точне розуміння локації помилки.
9. Що таке pytest?
   Сучасний, популярний та гнучкий фреймворк для написання й запуску автоматизованих тестів у Python.
10. Як pytest знаходить tests?
    Шукає файли test_*.py або *_test.py та функції/методи, що починаються з test_.
11. Якою командою запускається pytest?
    Командою pytest або python -m pytest у терміналі.
12. Для чого pytest -v?
    Прапорець -v (verbose) вмикає детальний вивід результатів для кожного тесту.

13. Для чого pytest -x?
    Прапорець -x зупиняє виконання тестової сесії одразу після першого упалого тесту.

14. Що означає Arrange–Act–Assert?
    Класичний паттерн структури тесту: Arrange (підготовка даних), Act (виконання дії), Assert (перевірка результату).

15. Для чого використовується assert?
    Для перевірки істинності тестової умови. Якщо умова False, генерується AssertionError і тест вважається проваленим.

16. Що таке assertion failure?
    Помилка, що виникає, коли фактичний результат тесту не відповідає очікуваному.

17. Чому floats не завжди слід порівнювати через ==?
    Через особливості представлення дійсних чисел у двійковій системі (помилки округлення).

18. Для чого pytest.approx?
    Для порівняння чисел із плаваючою крапкою з урахуванням допустимої погрешності (tolerance).

19. Для чого pytest.raises?
    Для перевірки того, що код викидає очікуваний виняток (exception).

20. Як перевірити exception message?
    Зберегти контекст у змінну with pytest.raises(...) as exc_info: і перевірити str(exc_info.value).

21. Як отримати exception object через exc_info?
    Через атрибут exc_info.value.

22. Що таке boundary testing?
    Метод тестування граничних значень допустимих вхідних діапазонів (наприклад, -1, 0, 1).

23. Які boundaries потрібно тестувати для діапазону 0–100?
    Значення: -1, 0, 1, 99, 100, 101.

24. Що таке parametrization?
    Запуск одного й того ж тесту з різними наборами вхідних даних та очікуваних результатів.

25. Для чого pytest.mark.parametrize?
    Декоратор для передачі списку аргументів у тестову функцію.

26. Що дають ids параметризованих tests?
    Зрозумілі назви для кожного набору даних у звіті запуску тестів.

27. Що таке fixture?
    Функція, яка створює та надає фіксоване середовище, об'єкти або дані для тестів.

28. Для чого використовуються fixtures?
    Для усунення дублювання коду підготовки (setup) та очищення (teardown) тестового оточення.

29. Що означає fixture setup?
    Код фікстури, що виконується *до* запуску тесту.

30. Що означає fixture teardown?
    Код фікстури, що виконується *після* завершення тесту (після оператора yield).

31. Для чого yield у fixture?
    Для розділення фаз setup і teardown у фікстурі.

32. Які fixture scopes підтримує pytest?
    function (default), class, module, package, session.

33. Який scope є default?
    function (фікстура створюється заново для кожного тесту).

34. Для чого conftest.py?
    Файл для збереження спільних фікстур та конфігурації pytest, які автоматично доступні у всіх тестах каталогу.

35. Чому fixtures із conftest.py не імпортують вручну?
    pytest знаходить і підключає їх автоматично за назвою аргументу функції.

36. Що таке tmp_path?
    Вбудована фікстура pytest, яка надає унікальну тимчасову директорію (pathlib.Path) для кожного тесту.

37. Чому tmp_path корисний?
    Він забезпечує ізоляцію файлових операцій і автоматичне видалення тестових файлів.

38. Що таке test isolation?
    Принцип, за яким тести не повинні впливати один на одного і можуть запускатися у довільному порядку.

39. Для чого monkeypatch?
    Вбудована фікстура для безпечної тимчасової модифікації системних змінних, системних атрибутів або функцій під час тесту.

40. Як monkeypatch змінює environment variable?
    За допомогою методу monkeypatch.setenv("VAR_NAME", "value").

41. Як видалити environment variable у test?
    За допомогою методу monkeypatch.delenv("VAR_NAME", raising=False).

42. Як підмінити function через monkeypatch?
    За допомогою monkeypatch.setattr(target, name, value).

43. Що таке test double?
    Загальний термін для будь-якої сурогатної підміни реального об'єкта в тестах (Dummy, Stub, Mock, Fake, Spy).

44. Що таке Dummy?
    Об'єкт, який передається в параметр, але ніколи фактично не використовується.

45. Що таке Stub?
    Об'єкт, що повертає заздалегідь заготоване значення на визначені виклики.

46. Що таке Fake?
    Об'єкт із працюючою, але спрощеною реалізацією (наприклад, InMemoryRepository).

47. Що таке Mock?
    Спеціальний об'єкт, який реєструє всі виклики до себе і дозволяє перевіряти взаємодію (interaction testing).

48. Що таке Spy?
    Огортка навколо реального об'єкта, яка записує інформацію про його виклики.

49. Для чого Mock?
    Для ізоляції тесту від зовнішніх залежностей (мережі, БД, API) та перевірки фактів виклику методів.

50. Що таке spec?
    Параметр Mock(spec=Class), який обмежує інтерфейс моку лише існуючими атрибутами реального класу.

51. Для чого return_value?
    Вказує, яке значення має повернути мок-функція при виклику.

52. Для чого side_effect?
    Дозволяє задати генерацію винятку або динамічну обробку під час виклику моку.

53. Як перевірити, що method викликано один раз?
    Викликати mock.method.assert_called_once().

54. Як перевірити parameters виклику?
    Викликати mock.method.assert_called_with(...) або assert_called_once_with(...).

55. Як перевірити, що method не викликався?
    Викликати mock.method.assert_not_called().

56. Що таке state testing?
    Тестування стану об'єкта (перевірка значень його атрибутів) після виконання дій.

57. Що таке interaction testing?
    Тестування фактів і параметрів взаємодії між об'єктами (чи був викликаний метод, скільки разів і з якими аргументами).

58. Коли interaction testing виправдане?
    Коли компонент викликає зовнішній сервіс (надсилання email, оплата, запит до стороннього API) і не має збереженого внутрішнього стану.

59. Що таке patch?
    Механізм із unittest.mock для тимчасової підміни об'єкта за його імпортованим шляхом.

60. Чому patch потрібно виконувати там, де name використовується?
    Тому що Python імпортує посилання на об'єкти; підміна має відбутися в модулі, де цей об'єкт викликається, а не де декларується.

61. Що таке AsyncMock?
    Спеціальний мок для підміни корутин та асинхронних функцій (async/await).

62. Як перевірити awaited call?
    За допомогою методів async_mock.assert_awaited(), assert_awaited_once_with().

63. Що таке flaky test?
    Тест, який то проходить, то падає без внесення змін у код (через недетермінованість, мережу або стан).

64. Чому flaky tests небезпечні?
    Вони підривають довіру до всієї системи автоматизованого тестування.

65. Що таке deterministic test?
    Тест, який за однакових умов завжди дає 100% однаковий результат.

66. Чому unit test не повинен залежати від Internet?
    Зовнішня мережа повільна, нестабільна та робить тести залежними від сторонніх серверів.

67. Коли в integration test доцільний real filesystem?
    Коли тестується безпосередній модуль зчитування/запису файлів (наприклад, CSV імпорт чи JSON експорт).

68. Чому не потрібно mock-ати все?
    Надмірний мокінг (*over-mocking*) призводить до того, що тести перевіряють самі моки, а не реальний бізнес-код.

69. Що таке over-mocking?
    Антипаттерн, коли у тесті замокано занадто багато внутрішніх класів, через що тест стає крихким і прив'язаним до деталей реалізації.

70. Що таке coverage?
    Метрика, яка вимірює відсоток вихідного коду програми, який був виконаний під час запуску тестів.

71. Що таке statement coverage?
    Відсоток виконаних інструкцій (рядків коду).

72. Що таке branch coverage?
    Відсоток виконаних гілок розгалуження (умов if/else, циклів).

73. Чим вони відрізняються?
    Statement coverage рахує лише відвідані рядки; branch coverage перевіряє, чи були пройдені всі можливі шляхи розгалужень умов.

74. Як отримати coverage через pytest-cov?
    Запустити pytest --cov=package_name.

75. Для чого --cov-report=term-missing?
    Виводить у консоль номери рядків коду, які не були виконані жодним тестом.

76. Для чого HTML coverage report?
    Надає зручний та інтерактивний веб-інтерфейс із підсвічуванням покритих/непокритих рядків безпосередньо у коді.

77. Чи гарантує 100% coverage відсутність bugs?
    Ні. Покриття показує лише те, що код був виконаний, але не гарантує правильність використаних тверджень (assert).

78. Чому meaningful assertions важливіші за coverage percentage?
    Тому що якісний тест має перевіряти бізнес-логіку та граничні умови, а не просто "пробігати" по рядках коду.

79. Що таке coverage threshold?
    Мінімально допустимий відсоток покриття коду, при недосягненні якого CI/CD збірка вважається провaленою.

80. Чому tests повинні бути незалежними один від одного?
    Щоб їх можна було запускати в довільному порядку, паралельно та без ризику взаємного впливу побічних ефектів.

Висновок

У ході виконання лабораторної роботи №6 було успішно розроблено та реалізовано повну систему автоматизованого тестування для застосунку обліку складських запасів (Варіант №10).

Під час роботи було створенно 48 автоматизованих тестів (з яких 45 модульних та 3 інтеграційних), що повністю покривають усі бізнес-сценарії, валідації, граничні значення, файлові IO-операції та асинхронні процедури. Завдяки використанню плагіна pytest-cov досягнуто показника покриття коду у 96.99% (із врахуванням *branch coverage*), що суттєво перевищує встановлений поріг у 80%.

У ході тестування було засвоєно роботу з фікстурами (conftest.py, yield, tmp_path), параметризацією тестів (@pytest.mark.parametrize), механізмами мокінгу (Mock, AsyncMock, return_value, side_effect) та підміни середовища (monkeypatch). Набуто практичних навичок аналізу непокритих гілок коду та виявлення прихованих дефектів.